



PMR3502 - Elementos de Robótica

Teleoperação

Grupo 1

Mateus Bonelli Salomao

Nicolas Helio Cunha Nabrink

Tomás Windisch Olenscki

São Paulo

2026

Sumário

1	Funcionamento do programa de teleoperação
----------	--

1 Funcionamento do programa de teleoperação

Nesta atividade, foi implementado o controle dos motores do robô a partir de uma interface web de teleoperação. A página web envia comandos por meio de joysticks virtuais, representados pelas variáveis *l* e *r*, correspondentes respectivamente ao motor esquerdo e ao motor direito. Esses valores são recebidos pelo programa em Python e convertidos em sinais elétricos aplicados à ponte H, permitindo controlar o sentido de rotação e a velocidade de cada motor.

O código utilizado no arquivo `motorctrl.py` é apresentado a seguir.

```
import gpiod
from gpiod.line import Direction, Value
from rpi_hardware_pwm import HardwarePWM
import contextlib
import numpy as np

# Cria um controlador de motor.
# Este gerador é decorado com contextmanager para garantir
# que o estado do gpio e pwm seja resetado na saída de escopo
@contextlib.contextmanager
def cria_controle_motor():
    # Configuração inicial: output com nível inicial baixo
    print("Criando controle de motores...", end="")
    confoutlow = gpiod.LineSettings(
        direction=Direction.OUTPUT,
        output_value=Value.INACTIVE
    )

    with gpiod.Chip("/dev/gpiochip0") as chip0:
        # Cria acesso às linhas de GPIO 5 e 6
        with chip0.request_lines({5: confoutlow, 6: confoutlow}) as linhas:
            pwma = HardwarePWM(0, 20, 0)
            pwmb = HardwarePWM(1, 20, 0)
            motor = motorCtrl(linhas, 5, 6, pwma, pwmb)
            print("Feito.")

        try:
            yield motor
        finally:
            print("Encerrando controle de motores...", end="")
            pwma.stop()
            pwmb.stop()
            print("Feito.")
```

```
class motorCtrl():
    def __init__(self, linhas, canal_direita,
                  canal_esquerda, pwma, pwmb):

        # Acesso a linhas gpio por gpiod
        self._linhas = linhas

        # Número do canal de GPIO da fase "B" do motor da direita
        self._cdireita = canal_direita

        # Número do canal de GPIO da fase "B" do motor da esquerda
        self._cesquerda = canal_esquerda

        # PWM conectado à fase "A" do motor da direita
        self._pwma = pwma

        # PWM conectado à fase "A" do motor da esquerda
        self._pwmb = pwmb

        self._pwma.start(0)
        self._pwmb.start(0)

    def set_lr(self, l, r):
        r = np.clip(r, -100, 100)
        l = np.clip(l, -100, 100)

        if l >= 0: # left motor forward
            self._pwmb.change_duty_cycle(l)
            self._linhas.set_value(self._cesquerda, Value.INACTIVE)

        elif l < 0: # move left motor backwards
            self._pwmb.change_duty_cycle(l + 100)
            self._linhas.set_value(self._cesquerda, Value.ACTIVE)

        if r >= 0: # right motor forward
            self._pwma.change_duty_cycle(r)
            self._linhas.set_value(self._cdireita, Value.INACTIVE)

        elif r < 0: # move right motor backwards
            self._pwma.change_duty_cycle(r + 100)
            self._linhas.set_value(self._cdireita, Value.ACTIVE)

# GPIO 5 - input B of motor A (right); controls direction
# GPIO 6 - input B of motor B (left); controls direction
```

```
# pin 32 - input A of motor A (PWM signal);  
# controls duty cycle -> angular speed  
# pin 33 - input A of motor B (PWM signal);  
# controls duty cycle -> angular speed  
  
# out B Low and out A high -> motor rotates 'forward'
```

A parte implementada foi o método `set_lr(self, l, r)`. Esse método recebe os comandos vindos dos joysticks virtuais e define o valor de PWM aplicado a cada motor. O valor `l` controla o motor esquerdo e o valor `r` controla o motor direito. Cada valor pode variar de -100 a 100, em que o sinal indica o sentido de rotação e o módulo indica a intensidade do comando.

As primeiras linhas implementadas no método são:

```
r = np.clip(r, -100, 100)  
l = np.clip(l, -100, 100)
```

Essas linhas foram adicionadas após percebermos um problema em que a conexão web com a Raspberry Pi caía ao utilizar velocidades máximas em qualquer um dos motores. Elas garantem que os valores recebidos permaneçam dentro do intervalo permitido. Assim, mesmo que a interface web envie algum valor fora da faixa esperada, o programa limita automaticamente o comando entre -100 e 100.

Para o motor esquerdo, o código verifica inicialmente se `l >= 0`. Nesse caso, o motor deve girar no sentido considerado positivo, isto é, para frente. Para isso, o PWM do motor esquerdo recebe diretamente o valor de `l` como ciclo de trabalho, e a entrada digital associada ao sentido de rotação é colocada em nível baixo:

```
self._pwmb.change_duty_cycle(l)  
self._linhas.set_value(self._cesquerda, Value.INACTIVE)
```

Quando `l < 0`, o motor esquerdo deve girar no sentido contrário. Nesse caso, a linha GPIO associada ao motor esquerdo é colocada em nível alto, alterando a configuração da ponte H. Com essa configuração, o acionamento do motor no sentido reverso ocorre durante a parcela do ciclo em que o sinal PWM está em nível baixo. Por isso, o valor aplicado ao PWM não é simplesmente o módulo de `l`, mas sim o valor complementar `l + 100`. Como `l` é negativo, essa expressão transforma o intervalo de comando negativo em um ciclo de trabalho adequado para o funcionamento reverso. Por exemplo, se `l = -100`, o ciclo aplicado é 0%, produzindo velocidade máxima para trás; se `l = -50`, o ciclo aplicado é 50%, produzindo aproximadamente meia velocidade; e, se `l = -10`, o ciclo aplicado é 90%, produzindo baixa velocidade para trás.

```
self._pwmb.change_duty_cycle(l + 100)  
self._linhas.set_value(self._cesquerda, Value.ACTIVE)
```

A mesma lógica é aplicada ao motor direito, usando a variável `r`. Quando `r >= 0`, o motor direito gira para frente, com a linha digital em nível baixo e o ciclo de trabalho igual a `r`. Quando `r < 0`, a linha digital é colocada em nível alto, invertendo o sentido de rotação, e o PWM recebe o valor `r + 100`.